

Chess++

Class **Game**

Member Variables :

- command_line_
- board_
- player_white_
- player_black
- active_player
- is_running_
- round_
- turn_count_
- move_history_

Methods :

- + checkMagicNumber(std:string& config_file_path) – checks if the configuration file starts with the correct magic number
- + loadConfigFile(std:string& config_file_path) – opens the file and reads its contents creating and placing pieces and items
- + validateCommand(Command command) – checks if the command is valid, returns errors if not
- + executeCommand(Command command) – calls the execute function of the corresponding command
- + moveHistory() – prints the history of all moves made so far
- + start() – handles the game logic calling the necessary command in order
- + endGameDraw() – checks the conditions for a draw and ends the game
- + endGameWin() – checks the conditions for a win and ends the game
- + getters, setters, constructor and destructor as needed

Class **Board**

Member Variables :

- squares_

Methods :

- + getSquare(Coordinates coordinates) – returns the pointer to the square
- + checkMove(Coordinates from, Coordinates to) – checks if the path from one square to another is clear and a piece can be moved, return true if it is possible otherwise returns false
- + placePiece(Coordinates coordinates, Piece* piece) – places a piece on a square with the given coordinates, return true if it is possible otherwise returns false

- + withinBounds(Coordinates coordinates) – checks if the given coordinates are within the bounds of the board, meant to be called in placePiece to make the checking of the conditions easier, return true if they are otherwise returns false
- + isPathClear(Coordinates from, Coordinates to) – checks if the squares along the given path are not occupied with other pieces, also a helper function, return true if they aren't otherwise returns false
- + getters, setters, constructor and destructor as needed

Class **Coordinates**

Member Variables :

- file_
- rank_

Methods :

- + convertFromString(std:string coordinates) – converts the coordinates, that are given as a string originally in the file, to a size_t and then they are assigned to file_ and rank_ accordingly
- + isValid(std:string coordinates) – checks if the coordinates are A through H and 1 through 8
- + distanceTo(Coordinates square_coordinates) – calculates the distance based on the equation, needed for executing certain effects
- + getters, setters, constructor and destructor as needed

Class **Move**

Member Variables :

- round_
- color_
- to_
- eaten_
- potion_
- piece_short_name
- potion_short_name

Methods :

- + print() – prints the information about the move
- + getters, setters, constructor and destructor as needed

Abstract class **Square** // da li treba da bude apstraktna

Member Variables :

- coordinates_
- item_
- piece_
- under_attack
- square_type

Methods :

- + isOccupied() – checks if a square is already occupied, return true if it is and false otherwise
- + getPiece() – returns a pointer to a piece
- + setPiece(Piece piece) – sets a piece on a square
- + removePiece(Piece piece) - removes a piece from a square
- + getItems() – returns a pointer to an item that's on a square
- + triggerEffect() – triggers an effect on a certain square, pure virtual method
- + getters, setters, constructor and destructor as needed

Subclasses **Mana Square, Boost Square, Spawn Square**

Methods :

- + triggerEffect() – triggers a specific effect of a square subclass, overriding a method from square
- + getters, setters, constructor and destructor as needed

Abstract class **Item**

Member Variables :

- id_
- display_name_
- type_

Methods :

- + getters, setters, constructor and destructor as needed

Subclasses **Tool, Potion**

Member Variables :

- tool_type_ / potion_type_

Methods :

- + getters, setters, constructor and destructor as needed

Abstract class **Piece**

Member Variables :

- owner_color_
- piece_type_
- id_
- short_name_
- value_
- power_
- item_
- is_frozen_
- coordinates_

Methods :

- + move() – virtual function that moves a piece
- + getters, setters, constructor and destructor as needed

Subclasses **King, Queen, Rook, Pawn, Bishop, Knight**

Member Variables :

- king_type_ / queen_type_ / rook_type_ / pawn_type_ / bishop_type_ / knight_type_

Methods :

- + triggerEffect() – triggers an effect of a piece
- + move() – moves a piece in a certain way specific to a piece type
- + getters, setters, constructor and destructor as needed

Class **Player**

Member Variables :

- color_
- mana_
- elo_
- pieces_
- prison_pieces_

Methods :

- + checkForCheck() – checks if a player is in danger of a check
- + checkForStaleMate() – check for a possibility of a stalemate
- + checkForCheckmate() – checks if a player is in a checkmate
- + addToPrison(Piece* piece) – adds players pieces to prison
- + printPrison() – prints information about a players prison
- + getters, setters, constructor and destructor as needed

Class **CommandLine**

Member Variables :

- parameter_counts_
- error_message_mapping_

Methods :

- + printPrompt() – prints the prompt for a user to write input
- + printErrorMessages() – prints all the error messages that may occur during user input
- + getCommand() – gets a command from the command line, the first part of the input
- + getters, setters, constructor and destructor as needed

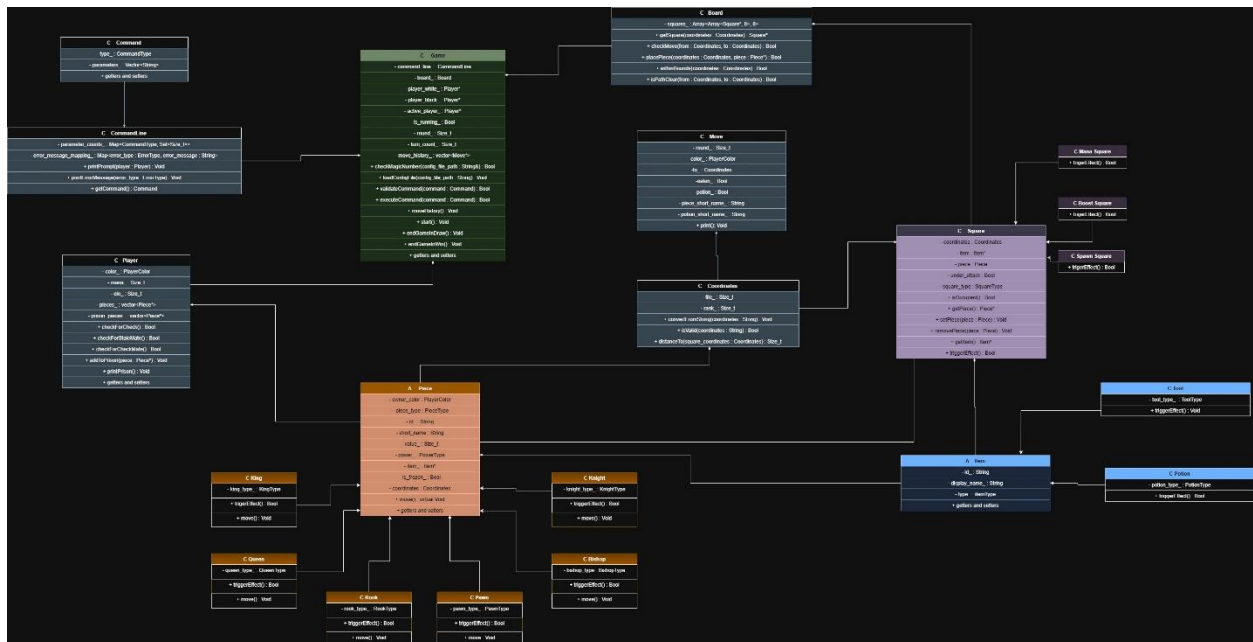
Class Command

Member Variables :

- type_
- parameters_

Methods :

- + getters, setters, constructor and destructor as needed



UML diagram